TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

KIẾN TRÚC MÁY TÍNH

LAB 01

LÀM QUEN MARS: XỬ LÝ I/O và CẤU TRÚC ĐIỀU KIỆN

Họ và tên	Nguyễn Đắc Anh Hào	**MSSV**	25520506
Lớp	VMTN2025	**Ngày thực hành**	06/10/2026
Tự chấm		**Link git**	

1. Tổng quan buổi thực hành

Sinh viên dùng trình mô phỏng **MARS 4.5** để biến kiến thức Chương 2 (kiến trúc bộ lệnh MIPS: toán hạng, số có dấu/không dấu, biểu diễn lệnh, lệnh điều kiện và nhảy) thành các thí nghiệm dự đoán – lập trình – quan sát – giải thích. Tình huống xuyên suốt: một chương trình tính tiền điện sinh hoạt theo biểu giá bậc thang, kèm một đoạn giai điệu ngắn phát bằng syscall MIDI để báo hoàn tất.

Sau buổi thực hành, Sinh viên có thể:

- Soạn, Assemble, chạy từng bước và gỡ lỗi chương trình MIPS32 trên MARS; dùng syscall nhập/xuất, thời gian hệ thống và MIDI.
- Phân biệt dòng lệnh nguồn, giả lệnh và lệnh cơ bản; đọc cột Code/Basic của Text Segment và tự giải mã các trường của lệnh dạng R, dạng I và độ lệch của lệnh nhánh.
- Hiện thực cấu trúc điều kiện và vòng lặp bằng `slt`/`sltu`/`beq`/`bne`; truy cập mảng trong bộ nhớ bằng địa chỉ cơ sở và chỉ số.
- Phân biệt so sánh có dấu và không dấu khi kiểm tra miền giá trị của dữ liệu nhập.
- Dùng Instruction Counter để đo số lệnh thực thi, xây dựng mô hình chi phí theo dữ liệu vào và đánh giá một thay đổi thiết kế bằng số liệu đo.

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen MARS, kiểm tra môi trường	15 phút	
TN 1	Từ lệnh nguồn đến mã máy: giả lệnh, Text Segment, Instruction Counter	20 phút	1,5
TN 2	Tính tiền điện bậc thang: nhập có kiểm tra, biểu giá lưu trong bộ nhớ	70 phút	4,0
TN 3	Phát giai điệu bằng syscall MIDI và đo thời gian chạy	25 phút	2,0
TN 4	Đo số lệnh và giải mã lệnh trên chương trình của mình	30 phút	2,0
Kết	Tổng kết, phản tư, nộp bài	20 phút	0,5
Tổng		**180 phút**	**10,0/10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- **Tham số cá nhân.** `a` là chữ số cuối của MSSV, `b` là chữ số liền trước. Mọi số liệu riêng của bài (đơn giá, nhịp độ, nhạc cụ, giá trị kiểm thử) được tính từ `a` và `b`; điền vào Bảng 1 trước khi bắt đầu.
- **Quy ước mã nguồn.** Mỗi chương trình có `.data`, `.text`, nhãn `main` và kết thúc bằng syscall 10. Chuỗi in ra và chú thích dùng ASCII không dấu. Ghi chú vai trò thanh ghi theo quy ước: `$t*` giá trị tạm, `$s*` giá trị cần giữ qua syscall, `$v0` số hiệu syscall, `$a0`–`$a3` đối số.
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi chạy trên MARS; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.

PHẦN 0 — Làm quen MARS (15 phút)

Giảng viên hướng dẫn nhanh giao diện MARS. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng dưới đây (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☐	Tạo file mới, lưu với tên `Lab01_<MSSV>_dien.asm`	Mỗi chương trình nộp là một file riêng; xem quy ước ở cuối tài liệu.
☐	Assemble (Run ▸ Assemble, F3), chạy (Run ▸ Go, F5), chạy từng lệnh (F7), lùi một lệnh (F8), đặt lại (F12)	Sau khi sửa mã phải Assemble lại. Lỗi Assemble hiện ở tab Mars Messages.
☐	Kiểm tra Settings: bỏ chọn **Delayed branching**; chọn **Permit extended (pseudo) instructions and formats**	Hai thiết lập này ảnh hưởng trực tiếp đến mã máy và số lệnh đếm được.
☐	Đọc các vùng của tab Execute: Text Segment (Address, Code, Basic, Source), Data Segment, Registers	Tích **Labels** ở dưới Data Segment để xem địa chỉ của nhãn dữ liệu.
☐	Đặt breakpoint (ô Bkpt) và chạy từng bước để quan sát thanh ghi đổi màu	Thanh ghi vừa bị ghi sẽ được tô sáng.
☐	Mở Tools ▸ Instruction Counter và bấm **Connect to MIPS** trước khi chạy	Công cụ chỉ đếm khi đã kết nối; bấm **Reset** để đếm lại từ 0.
☐	Tra bảng syscall trong Help ▸ Help (thẻ MIPS, mục Syscalls)	Bảng tóm tắt các syscall dùng trong buổi này nằm ngay dưới.
☐	Chụp màn hình cửa sổ MARS và dán vào khung ảnh của file này	Chụp cả vùng Run I/O lẫn vùng mã đang xét.

Các syscall và lệnh dùng trong buổi này

Đặt số hiệu dịch vụ vào `$v0`, đối số vào `$a0`–`$a3`, rồi thực hiện `syscall`.

$v0	Dịch vụ	Đối số	Kết quả
1	In số nguyên	`$a0` = giá trị	
4	In chuỗi kết thúc bằng 0	`$a0` = địa chỉ chuỗi	
5	Đọc số nguyên có dấu		`$v0` = giá trị đọc được
10	Kết thúc chương trình		
30	Lấy thời gian hệ thống (ms)		`$a0` = 32 bit thấp, `$a1` = 32 bit cao
31	Phát một nốt MIDI, trả về ngay	`$a0` cao độ (0–127), `$a1` độ dài (ms), `$a2` nhạc cụ (0–127), `$a3` âm lượng (0–127)	
32	Chờ (ngủ)	`$a0` = số ms	
33	Phát một nốt MIDI, chờ đến khi nốt kết thúc	như syscall 31	

Lệnh `mul rd, rs, rt` nhân hai số và ghi 32 bit thấp của tích vào `rd` (cơ chế bên trong sẽ học ở Chương 3, ở đây dùng như một lệnh có sẵn).

Mục	Ghi nhận
Phiên bản MARS đang dùng (Help ▸ About)	MARS 4.5
Máy có phát được âm thanh qua MIDI không? (Có / Không)	Có

Bước 0 – Kiểm tra nhanh. Tạo chương trình in dòng chữ `Xin chao MARS` rồi kết thúc (chỉ dùng syscall 4 và 10). Assemble, chạy và dán ảnh chụp vùng Run I/O vào khung dưới. File này chỉ để làm quen, không nộp.

Dán ảnh chụp cửa sổ MARS sau khi chạy chương trình kiểm tra (thấy rõ dòng in ra)

Tham số cá nhân

Điền `a`, `b` rồi tính các giá trị còn lại. Các giá trị này dùng cho TN 2 và TN 3.

Bảng 1. Tham số cá nhân

Đại lượng	Cách tính	Giá trị của Sinh viên
a	Chữ số cuối của MSSV	6
b	Chữ số liền trước chữ số cuối của MSSV	0
Đơn giá bậc 1 (đồng/kWh)	1600 + 20*a	1720
Số nửa cung dịch giọng	a	6
Độ dài một đơn vị nhịp (ms)	150 + 10*b	150
Nhạc cụ (số hiệu chương trình MIDI)	Phần dư của b khi chia 4 là 0, 1, 2, 3 tương ứng 0, 24, 40, 73	0
Số kWh kiểm thử cá nhân x_b	100 + 25*b	100

THÍ NGHIỆM 1 — Từ lệnh nguồn đến mã máy

Thời lượng gợi ý: 20 phút · File: tn1.asm (không nộp) · Điểm: 1,5

Tình huống

Trước khi viết chương trình lớn, kỹ sư cần biết một dòng mã nguồn thực sự trở thành bao nhiêu lệnh máy: số lệnh này quyết định kích thước chương trình và (cùng với số lệnh thực thi) thời gian chạy. Thí nghiệm này khảo sát một chương trình nhỏ để làm rõ ba con số: số dòng nguồn, số lệnh cơ bản trong bộ nhớ và số lệnh thực sự được thực thi.

Chương trình khảo sát tn1.asm (gõ lại nguyên văn, chưa Assemble)

```
        .data
msg:    .asciiz "Ket qua: "
        .text
main:   li   $t0, 25
        li   $t1, 70000
        add  $t2, $t0, $t1
        blt  $t0, $t1, small
        li   $t2, 0
small:  la   $a0, msg
        li   $v0, 4
        syscall
        move $a0, $t2
        li   $v0, 1
        syscall
        li   $v0, 10
        syscall
```

Bước 1.1 – Dự đoán trước khi Assemble. Chưa chạy MARS. Đọc chương trình và điền cột Dự đoán của Bảng 2 và Bảng 3.

DỰ ĐOÁN Mỗi dòng nguồn ở Bảng 2 sinh ra bao nhiêu lệnh cơ bản? Hai lệnh trong Bảng 3 là bao nhiêu dòng nguồn, lệnh cơ bản (trong bộ nhớ) và lệnh được thực thi? Chương trình in ra gì?

Bước 1.2 – Assemble và quan sát. Assemble, đọc Text Segment, điền cột Quan sát. Mở Instruction Counter (Connect to MIPS), chạy chương trình và đọc tổng số lệnh.

QUAN SÁT Cột Basic cho biết các lệnh cơ bản thực sự nằm trong bộ nhớ; cột Source là dòng nguồn tương ứng.

Bảng 2. Số lệnh cơ bản sinh ra từ mỗi dòng nguồn

Dòng nguồn	Dự đoán (số lệnh)	Quan sát (số lệnh)	Các lệnh cơ bản quan sát được (cột Basic)
`li $t0, 25`	1	1	addiu
`li $t1, 70000`	1	2	lui, ori
`blt $t0, $t1, small`	2	2	slt, bne
`la $a0, msg`	2	2	lui, ori
`move $a0, $t2`	1	1	addu

Bảng 3. Số dòng lệnh nguồn, lệnh cơ bản, lệnh được thực thi của chương trình khảo sát

Đại lượng	Dự đoán	Quan sát	Nguồn đọc kết quả
Số dòng lệnh nguồn trong `.text`	13	13	Đếm trong Source
Số lệnh cơ bản trong bộ nhớ (tĩnh)	16	16	Số hàng của Text Segment
Số lệnh được thực thi (động)	15	15	Instruction Counter, mục Total
Chuỗi in ra Run I/O	Ket qua: 70025	Ket qua: 70025	Cửa sổ Run I/O

Câu 1.1 (Hiểu). Vì sao `li $t0, 25` chỉ cần một lệnh cơ bản còn `li $t1, 70000` cần hai? Giá trị 70000 không vừa trường nào của lệnh dạng I? Thanh ghi `$at` (số 1) xuất hiện ở đâu trong Text Segment và đóng vai trò gì?

Trả lời: li $t0, 25 chỉ cần 1 lệnh cơ bản vì 25 biểu diễn được trong trường immediate 16 bit. Số 70000 không vừa trường immediate 16 bit của lệnh dạng I. Vì vậy, cần tách thành 2 phần: 70000 = 1 x 65536 + 4464 = 0x000011170. Thanh ghi $at là thanh ghi tạm dành cho assembler, xuất hiện trong phần triển khai. Trong Text Segment, nó xuất hiện ở các lệnh cơ bản sinh ra từ li $t1, 70000, blt và la.

Câu 1.2 (Phân tích). Giải thích vì sao ba con số ở Bảng 3 khác nhau. Trong hai lệnh `blt` và `li $t2, 0`, lệnh nào góp phần làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ và vì sao?

Trả lời: Bởi vì 13 dòng nguồn là số câu lệnh người lập trình viết, 16 câu lệnh trong bộ nhớ là số lệnh cơ bản sau khi assembler mở rộng các lệnh giả, 15 lệnh thực thi là số lệnh thực sự được chạy theo luồng chương trình. Trong hai lệnh blt và li $t2, 0 thì lệnh li $t2, 0 là lệnh bị bỏ qua góp phần làm số lệnh thực thi nhỏ hơn số lệnh bộ nhớ

Bước 1.3 – Giải mã bằng tay. Đọc mã hex của lệnh `add $t2, $t0, $t1` ở cột Code **trước**, tách các trường rồi so với cột Basic. Làm tương tự với lệnh nhánh do `blt` sinh ra.

Bảng 4. Giải mã lệnh dạng R: `add $t2, $t0, $t1`

Địa chỉ	Mã hex (Code)	opcode	rs	rt	rd	shamt	funct
0x00400000C	0x01095020	000000	01000	01001	01010	00000	100000

Bảng 5. Giải mã lệnh nhánh sinh ra từ `blt` (lệnh thứ hai của cặp)

Địa chỉ	Mã hex (Code)	opcode	rs	rt	immediate (thập phân, có dấu)	Địa chỉ nhãn `small`
0x00400014	0x14200001	000101	00001	00000	1	0x0040001C

Câu 1.3 (Áp dụng). Viết phép tính cho thấy trường immediate của lệnh nhánh ở Bảng 5 được suy ra từ địa chỉ của nó và địa chỉ nhãn `small` (nêu rõ vì sao có số hạng 4 và phép chia 4). Nếu chèn thêm 3 lệnh cơ bản vào giữa `blt` và `small`, trường immediate đổi thành bao nhiêu?

Trả lời: immediate = [0x0040001C – (0x00400014 + 4)]/4 = 4/4 = 1. Lý do có số hạng 4 vì MIPS tính độ dời của nhánh từ địa chỉ lệnh kế tiếp, tức PC + 4. Khoảng cách tính có đơn vị byte, còn immediate có đơn vị word 4 byte nên phải chia 4. Nếu chèn thêm 3 lệnh cơ bản thì immediate = 16/4 = 4.

Bước 1.4 – Dự đoán – quan sát lần hai. Trong tn1.asm thay `blt` bằng `bgt` (giữ nguyên nhãn và các dòng còn lại).

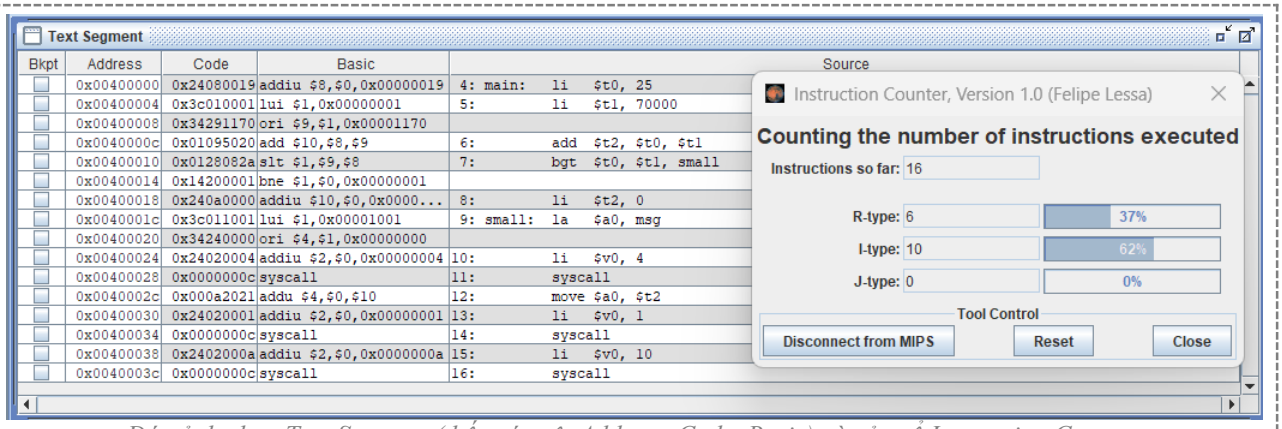
DỰ ĐOÁN Số lệnh thực thi bây giờ là bao nhiêu và chuỗi in ra là gì? Ghi dự đoán vào Bảng 6, Assemble, đặt lại Instruction Counter, chạy và ghi quan sát.

Bảng 6. Thay `blt` bằng `bgt`

Đại lượng	Dự đoán	Quan sát
Số lệnh được thực thi	16	16
Chuỗi in ra	Ket qua: 0	Ket qua: 0

Câu 1.4 (Phân tích). Ba lệnh `syscall` trong chương trình có cùng mã máy hay khác nhau (ghi mã hex quan sát được)? Nếu giống nhau thì máy phân biệt dịch vụ cần thực hiện bằng cách nào?

Trả lời: Ba lệnh syscall có cùng mã máy 0x0000000C. MARS chọn dịch vụ bằng giá trị thanh ghi $v0, còn $a0 chứa đối số tương ứng



Dán ảnh chụp Text Segment (thấy các cột Address, Code, Basic) và cửa sổ Instruction Counter

THÍ NGHIỆM 2 — Tính tiền điện bậc thang

Thời lượng gợi ý: 70 phút · File nộp: Lab01_<MSSV>_dien.asm · Điểm: 4,0

Tình huống

Một công ty điện lực cần chương trình tính tiền điện sinh hoạt của một hộ trong một tháng. Điện năng tiêu thụ được chia thành sáu bậc; **mỗi kWh được tính theo đơn giá của bậc chứa nó**, và tổng tiền là tổng của các phần ở từng bậc. Biểu giá dưới đây là biểu giá giả lập dùng cho bài thực hành. Chương trình chạy trên một vi xử lý nhúng nhỏ nên bảng giá phải nằm trong bộ nhớ dữ liệu để thay đổi khi biểu giá đổi mà không phải sửa phần lệnh.

Bảng 7. Biểu giá giả lập (đồng/kWh, số nguyên)

Bậc	Từ (kWh)	Đến (kWh)	Đơn giá (đồng/kWh)
1	0	50	1600 + 20*a
2	51	100	1660 + 20*a
3	101	200	1950 + 20*a
4	201	300	2500 + 20*a
5	301	400	2850 + 20*a
6	401	9999	3000 + 20*a

Đặc tả chương trình

- **Nhập.** In lời nhắc `Nhap so kWh (0..9999): ` rồi đọc một số nguyên có dấu bằng syscall 5.
- **Kiểm tra.** Giá trị hợp lệ khi $0 \le x \le 9999$. Nếu không hợp lệ, in `Gia tri khong hop le, nhap lai.` rồi nhắc nhập lại cho đến khi hợp lệ. Việc kiểm tra miền giá trị chỉ được dùng **một** lệnh so sánh và **một** lệnh nhánh.
- **Dữ liệu.** Ngưỡng các bậc và đơn giá lưu trong `.data` dưới dạng mảng `.word`. Phần tính toán duyệt các mảng này bằng vòng lặp; đổi giá trị trong mảng thì không phải sửa phần lệnh. Không viết riêng mã cho từng bậc.
- **Xuất.** In đúng khuôn `Tien dien: <so tien> dong` rồi xuống dòng, sau đó kết thúc chương trình.
- **Giao diện thanh ghi.** Ghi chú ở đầu file vai trò của từng thanh ghi `$s*` đã dùng.

Bước 2.1 – Dự đoán bằng tay. Chưa viết mã. Điền đơn giá 6 bậc của Sinh viên (từ Bảng 1) vào Bảng 8, rồi tính tay tiền điện cho các giá trị x ở Bảng 10 (có thể dùng máy tính bỏ túi).

DỰ ĐOÁN Ghi tiền điện dự kiến cho từng x. Chú ý các giá trị nằm đúng trên hoặc sát ranh giới bậc.

Bảng 8. Đơn giá 6 bậc theo tham số a của Sinh viên

Bậc	1	2	3	4	5	6
Đơn giá	1720	1780	2070	2620	2970	3120

Bước 2.2 – Thiết kế trước khi viết mã. Mô tả thuật toán bằng lời hoặc mã giả (không cần đúng cú pháp) và lập bảng phân công thanh ghi.

THIẾT KẾ Ghi rõ điều kiện kết thúc vòng lặp, cách xác định phần điện năng thuộc mỗi bậc và vì sao chương trình không cần biết trước số bậc đã dùng.

Thuật toán: Nhập số điện x. So sánh để kiểm tra x thuộc đoạn [0,9999], nếu không hợp lệ thì thông báo và nhập lại. Tạo điện năng còn lại bằng x, tổng tiền và cận dưới bằng 0, hai con trỏ ở đầu hai mảng. Lặp khi còn điện năng: lấy ngưỡng trên và đơn giá, tính lượng điện của bậc bằng min, cộng tiền và trừ lượng điện đã tính. Cập nhật cận dưới bằng ngưỡng trên, tăng 2 con trỏ 4 byte. Khi hết điện năng, in tổng tiền. Vòng lặp tự dừng nên không cần biết trước số bậc sử dụng.

Bảng 9. Phân công thanh ghi

Thanh ghi	Vai trò trong chương trình
$s0	Điện năng còn lại chưa tính tiền
$s1	Tổng tiền điện
$s2	Cận dưới của bậc hiện tại
$s3	Con trỏ vào mảng ngưỡng
$s4	Con trỏ vào mảng đơn giá

Bước 2.3 – Lập trình. Viết chương trình theo đặc tả trong file `Lab01_<MSSV>_dien.asm`. Assemble, sửa lỗi, chạy từng bước ít nhất một lần để theo dõi vòng lặp qua các bậc.

LẬP TRÌNH Khi gỡ lỗi, đặt breakpoint ở đầu thân vòng lặp và theo dõi các thanh ghi tổng tiền, cận dưới và con trỏ vào mảng.

Bước 2.4 – Kiểm thử và so sánh với dự đoán. Chạy chương trình lần lượt với các giá trị ở Bảng 10 và Bảng 11.

QUAN SÁT Ghi kết quả chương trình in ra và đối chiếu với dự đoán. Nếu lệch, tìm xem dự đoán tay hay chương trình sai, ghi kết luận vào cột cuối.

Bảng 10. Tiền điện: dự đoán và quan sát

x (kWh)	Dự đoán (đồng)	Quan sát (đồng)	Khớp? Nếu lệch, nguyên nhân
30	51600	51600	Có
50	86000	86000	Có
51	87780	87780	Có
120	216400	216400	Có
250	513000	513000	Có
401	944120	944120	Có
x_b = 100	175000	175000	Có

Bảng 11. Dữ liệu nhập không hợp lệ và giá trị biên của miền nhập

Dữ liệu nhập	Dự đoán hành vi	Quan sát
0	Hợp lệ, "Tien dien: 0 dong" rồi kết thúc	Hợp lệ, "Tien dien: 0 dong" rồi kết thúc
9999	Hợp lệ, "Tien dien: 30889880 dong" rồi kết thúc	Hợp lệ, "Tien dien: 30889880 dong" rồi kết thúc
10000	In "Gia tri khong hop le, nhap lai." Rồi yêu cầu nhập lại	In "Gia tri khong hop le, nhap lai." Rồi yêu cầu nhập lại
−1	In "Gia tri khong hop le, nhap lai." Rồi yêu cầu nhập lại	In "Gia tri khong hop le, nhap lai." Rồi yêu cầu nhập lại

Câu 2.1 (Phân tích). Chỉ rõ lệnh so sánh và lệnh nhánh dùng để kiểm tra miền [0, 9999] trong chương trình của Sinh viên. Giải thích vì sao **một** phép so sánh bắt được cả x < 0 lẫn x ≥ 10000.

Trả lời: Sử dụng cặp lệnh sltiu $t0, $s0, 10000 và bne $t0, $zero, hop_le. Phép so sánh không dấu coi giá trị trong $s0 là số nguyên không dấu 32 bit từ 0 đến 2^32-1. Khi 0 <= x < 10000, điều kiện đúng nên $t0 = 1. Khi x >= 10000, điều kiện sai nên $t0 = 0. Khi x < 0 (số âm trong biểu diễn bù 2), bit dấu (MSB) bằng 1 nên khi diễn giải không dấu, nó trở thành một số nguyên cực lớn (>= 2^31 = 2.147.483.648 > 10000), làm điều kiện nhỏ hơn 10000 cũng sai ($t0 = 0). Nhờ đó, chỉ một phép so sánh không dấu bắt được cả hai biên.

Bước 2.5 – Thí nghiệm có dấu và không dấu. Đổi lệnh so sánh ở câu 2.1 sang phiên bản so sánh **ngược** kiểu (có dấu thành không dấu hoặc ngược lại), Assemble lại, nhập x = −1.

DỰ ĐOÁN Chương trình sẽ làm gì? Ghi dự đoán trước, rồi chạy và ghi quan sát. Sau khi xong, khôi phục lệnh đúng.

Dự đoán	Quan sát
Chương trình coi -1 là hợp lệ, đi vào vòng lặp tính tiền và lặp vô hạn hoặc tính sai	Chương trình không báo lỗi, chạy thẳng vào vòng lặp tính tiền điện và lặp vượt ra ngoài phạm vi mảng dẫn đến kết quả sai

Câu 2.2 (Phân tích). Giải thích kết quả quan sát bằng cách xét biểu diễn bù hai của −1 và cách hai lệnh so sánh diễn giải cùng một chuỗi bit.

Trả lời: Trong hệ bù hai 32 bit, số -1 được biểu diễn bằng chuỗi bit 0xFFFFFFFF (32 bit 1). Lệnh có dấu slti diễn giải chuỗi bit này là giá trị -1, và vì -1 < 10000 nên trả về 1. Ngược lại, lệnh không dấu sltiu diễn giải cùng chuỗi bit 0xFFFFFFFF là 2^32 - 1 = 4.294.967.295, lớn hơn 10000 nên trả về 0.

Bước 2.6 – Quan sát bộ nhớ. Mở Tools → tích **Labels**, quan sát cửa sổ Labels và tìm địa chỉ nhãn của mảng đơn giá trong Data Segment. Đọc phần tử thứ 3 (chỉ số 2) ở dạng hex và thập phân.

QUAN SÁT Đối chiếu địa chỉ phần tử với địa chỉ nhãn và với lệnh nạp dữ liệu tương ứng trong chương trình.

Bảng 12. Mảng đơn giá trong Data Segment

Địa chỉ nhãn mảng	Địa chỉ phần tử thứ 3	Giá trị (hex)	Giá trị (thập phân)
0x10010064	0x1001006c	0x00000816	2070

Câu 2.3 (Áp dụng). Tính tổng tiền lớn nhất mà chương trình có thể in ra với miền nhập hợp lệ và cho biết phép cộng có thể gây tràn có dấu 32 bit hay không. Nếu miền nhập mở rộng đến N kWh thì N lớn nhất là bao nhiêu để tổng tiền vẫn không tràn? Nêu cách tính.

Trả lời: Tổng tiền lớn nhất trong miền hợp lệ ứng với x = 9999kWh là 30.889.880 đồng. Số nguyên có dấu 32 bit biểu diễn được tối đa đến 2^31 - 1 = 2.147.483.647. Vì 30.889.880 <<< 2.147.483.647 nên phép cộng không gây tràn số. Nếu mở rộng miền nhập đến N kWh (N > 400), tổng tiền ở 5 bậc đầu (400 kWh đầu) là: 50.1720 + 50.1780 + 100. 2070 + 100. 2620 + 100. 2970 = 941.000 đồng. Công thức tổng tiền cho N kWh là: T(N) = 941.000 + (N - 400). 3120. Để không tràn số có dấu 32 bit: 941.000 + (N - 400). 3120 <= 2.147.483.647 => N - 400 <= [2.146.542.647/3120] = 688.000 - 6 = 687.994. Suy ra N_max = 688.394 kWh.

Câu 2.4 (Đánh giá). So sánh thiết kế duyệt mảng của Sinh viên với cách viết sáu nhánh `if` riêng cho từng bậc: nêu ít nhất hai tiêu chí (ví dụ kích thước mã, khi biểu giá đổi, khi thêm bậc) và kết luận phương án nào hợp lý hơn cho thiết bị nhúng trong tình huống này.

Trả lời: Cách duyệt mảng chỉ dùng một vòng lặp cố định khoảng chục lệnh máy dù có bao nhiêu bậc, cách viết 6 nhánh if-else làm mã nguồn phình to gấp nhiều lần do lặp lại cấu trúc kiểm tra và nhân cộng ở từng nhánh. Với thiết kế duyệt mảng, khi nhà nước đổi giá điện hoặc tăng từ 6 bậc lên 8 bậc, chỉ cần cập nhật vùng nhớ dữ liệu (.data) mà không cần biên dịch lại hay sửa đổi logic lệnh (.text). Với 6 nhánh if, phải viết lại mã lệnh và nạp lại chương trình => Duyệt mảng tối ưu và phù hợp hơn.

*Dán ảnh chụp Run I/O với x = x_*



b của Sinh viên (thấy lời nhắc, giá trị nhập và dòng kết quả)

THÍ NGHIỆM 3 — Phát giai điệu bằng syscall MIDI

Thời lượng gợi ý: 25 phút · File nộp: Lab01_<MSSV>_nhac.asm · Điểm: 2,0

Tình huống

Thiết bị cần phát một giai điệu ngắn để báo hoàn tất. Giai điệu lưu dưới dạng hai mảng (cao độ và độ dài) trong bộ nhớ dữ liệu, chương trình duyệt mảng và gọi syscall MIDI cho từng nốt. Giai điệu là chủ đề *Ode to Joy* của L. van Beethoven (thuộc phạm vi công cộng); cao độ theo số hiệu nốt MIDI (60 là C4).

Bảng 13. Dữ liệu giai điệu (độ dài tính theo đơn vị nhịp, 1 đơn vị = giá trị `150 + 10*b` ms ở Bảng 1)

Nốt	Tên	Cao độ	Độ dài	Nốt	Tên	Cao độ	Độ dài
1	E4	64	2	9	C4	60	2
2	E4	64	2	10	C4	60	2
3	F4	65	2	11	D4	62	2
4	G4	67	2	12	E4	64	2
5	G4	67	2	13	E4	64	3
6	F4	65	2	14	D4	62	1
7	E4	64	2	15	D4	62	4
8	D4	62	2				

Đặc tả chương trình

- Hai mảng `.word` trong `.data` chứa cao độ và độ dài (đơn vị nhịp) của 15 nốt; số nốt cũng là một dữ liệu trong `.data`.
- Với mỗi nốt: cao độ thực phát bằng cao độ trong mảng cộng `a`; thời lượng (ms) bằng độ dài nhân với `150 + 10*b`; nhạc cụ và âm lượng 100 theo tham số cá nhân. In ra một dòng `<cao do> <ms>` rồi phát nốt bằng syscall 33.
- Đo thời gian chạy của cả vòng phát bằng syscall 30 (hiệu của hai lần đọc `$a0`). Cuối chương trình in `Tong du kien (ms): <so>` (chương trình tự cộng dồn thời lượng các nốt) và `Thoi gian do duoc (ms): <so>`.

Bước 3.1 – Dự đoán. Dùng dữ liệu của Sinh viên trước khi viết mã.

DỰ ĐOÁN Tổng thời lượng dự kiến của 15 nốt là bao nhiêu ms? Nếu thay syscall 33 bằng syscall 31 (giữ nguyên tham số), điều gì xảy ra với âm thanh và với thời gian đo được?

Tổng dự kiến (ms)	Dự đoán khi thay 33 bằng 31
4800	Syscall 31 phát nốt và trả về ngay lập tức mà không chờ nốt kết thúc, nên 15 nốt vang lên gần như cùng lúc và thời gian đo được rất nhỏ

Bước 3.2 – Lập trình và quan sát. Viết `Lab01_<MSSV>_nhac.asm` theo đặc tả, chạy và ghi số liệu vào Bảng 14. Nếu máy không phát được âm thanh, vẫn làm đủ các bước: ghi "Không nghe được" ở cột Nghe thấy và dựa vào các dòng in ra và thời gian đo được.

QUAN SÁT Nghe giai điệu có đúng nhịp không? Đối chiếu dòng in cuối với dự đoán ở Bước 3.1.

Bước 3.3 – So sánh ba cách phát. Thử lần lượt ba cấu hình và điền Bảng 14: (A) syscall 33; (B) syscall 31 thay cho 33; (C) syscall 31 rồi ngay sau đó syscall 32 với `$a0` bằng độ dài nốt (ms).

Bảng 14. Ba cấu hình phát nốt

Cấu hình	Nghe thấy	Thời gian đo được (ms)	Số dòng in ra	Nhận xét ngắn
A: 33	Giai điệu phát rõ từng nốt, đúng nhịp	16468	17	Phát đồng bộ, chờ hết nốt mới qua nốt tiếp theo.
B: 31	Tất cả 15 nốt kêu chồng lên nhau cùng lúc	2	17	Phát bất đồng bộ, vòng lặp kết thúc ngay lập tức.
C: 31 + 32	Giai điệu phát rõ từng nốt, đúng nhịp như A	4815	17	Syscall 31 phát bất đồng bộ kết hợp syscall 32 ngủ đúng bằng thời lượng nốt tạo hiệu ứng tương đương 33.

Câu 3.1 (Phân tích). Giải thích chênh lệch (nếu có) giữa tổng dự kiến và thời gian đo được ở cấu hình A. Liệt kê các nguồn gây chênh lệch mà Sinh viên nhận ra.

Trả lời: Thời gian đo được ở cấu hình A là 16468 ms, lớn hơn tổng dự kiến 4800 ms khoảng 11668 ms. Nguyên nhân có thể do chi phí xử lý và đồng bộ MIDI khi sử dụng syscall 33, thời gian xuất dữ liệu ra màn hình, thời gian thực thi các lệnh và độ trễ của hệ điều hành. Vì syscall 33 phải chờ nốt nhạc hoàn tất nên các độ trễ này được cộng vào tổng thời gian đo. Mức chênh lệch còn phụ thuộc vào máy tính và môi trường chạy MARS.

Câu 3.2 (Áp dụng). Nếu mảng cao độ khai báo bằng `.byte` thay vì `.word`, lệnh nạp phần tử và cách tính địa chỉ phần tử thứ i thay đổi thế nào? Chọn `lb` hay `lbu` cho trường hợp cao độ có thể lên đến 127 và cho trường hợp tổng quát 0–255? Vì sao?

*Trả lời: Nếu mảng khai báo bằng .byte, mỗi phần tử chỉ chiếm 1 byte thay vì 4 byte:Cách tính địa chỉ phần tử thứ i: Địa chỉ = địa_chỉ_cơ_sở + i * 1 (tăng con trỏ thêm 1 thay vì 4 sau mỗi vòng). Lệnh nạp phần tử: Dùng lb hoặc lbu thay cho lw. Với cao độ MIDI từ 0 đến 127 (bit thứ 7 luôn bằng 0): dùng lb hay lbu đều cho kết quả như nhau. Nhưng với trường hợp tổng quát 0-255: bắt buộc chọn lbu vì các giá trị từ 128 đến 255 có bit cao nhất (bit 7) bằng 1; nếu dùng lb, lệnh sẽ mở rộng dấu thành số âm 32 bit làm sai lệch giá trị cao độ.*

Câu 3.3 (Đánh giá). Dựa trên Bảng 14, thiết bị vừa phải phát nhạc vừa phải đọc cảm biến thì nên dùng cấu hình nào? Nêu một rủi ro khi dùng cấu hình B mà không chờ.

Trả lời: Thiết bị vừa phải phát nhạc vừa phải đọc cảm biến nên dùng cấu hình kết hợp nguyên lý của B/C (dùng syscall 31 kết hợp chia nhỏ thời gian chờ hoặc kiểm tra cảm biến xen kẽ trong lúc nốt nhạc đang ngân), vì syscall 33 làm treo hoàn toàn CPU trong suốt thời gian nốt nhạc kêu khiến thiết bị không thể phản hồi cảm biến kịp thời. Rủi ro khi dùng cấu hình B mà không chờ: Các nốt nhạc bị gửi dồn dập cùng một thời điểm làm chồng chéo âm thanh (mất giai điệu), hoặc làm tràn hàng đợi bộ đệm phần cứng âm thanh.

THÍ NGHIỆM 4 — Đo số lệnh và giải mã lệnh trên chương trình của mình

Thời lượng gợi ý: 30 phút · Dùng chương trình `Lab01_<MSSV>_dien.asm` đã hoàn thành · Điểm: 2,0

> **Tình huống**
> Trước khi triển khai lên vi xử lý nhúng có tài nguyên hạn chế, nhóm phát triển cần ước lượng chi phí của chương trình theo dữ liệu vào và hiểu rõ mã máy sinh ra. Sinh viên đo số lệnh thực thi của chương trình tính tiền điện, xây dựng mô hình chi phí đơn giản và giải mã một số lệnh của chính chương trình mình.

Bước 4.1 – Đo số lệnh thực thi. Mở Instruction Counter (Connect to MIPS). Với mỗi x ở Bảng 15, đặt lại bộ đếm, chạy, ghi tổng số lệnh thực thi. Đo x = 10 và x = 120 trước.

DỰ ĐOÁN Từ hai số đo đầu tiên, đề xuất công thức N(x) và điền cột Dự đoán cho các hàng còn lại; sau đó đo các hàng còn lại và so sánh.

Bảng 15. Số lệnh thực thi theo số kWh nhập

x (kWh)	Số bậc đã dùng hết	N dự đoán	N đo được	Lệch
10	1	—	43	—
120	3	—	69	—
50	1	42	42	0
250	4	82	82	0
401	6	108	108	0

Câu 4.1 (Phân tích). Viết công thức N(x) của chương trình Sinh viên và giải thích ý nghĩa từng hệ số bằng cách đếm số lệnh trong phần chuẩn bị, trong thân vòng lặp mỗi bậc và trong phần in kết quả. Vì sao N không đổi với mọi x thuộc cùng một bậc?

Trả lời: Với k là số bậc đã duyệt, số lệnh thực thi được tính theo công thức N(k) = 29 + 13k + δ, trong đó δ = 1 nếu bậc cuối chưa dùng hết và δ = 0 nếu x nằm đúng ngưỡng trên. Phần cố định gồm 15 lệnh nhập và khởi tạo, 1 lệnh kiểm tra thoát và 13 lệnh xuất kết quả. Mỗi bậc thực thi 13 lệnh cơ bản. N chủ yếu phụ thuộc vào số bậc duyệt chứ không phụ thuộc trực tiếp vào số kWh, ngoại trừ trường hợp x nằm đúng ngưỡng bậc.

Câu 4.2 (Đánh giá). Nếu biểu giá có 60 bậc và x nằm ở bậc cuối, N xấp xỉ bao nhiêu? Đề xuất **một** thay đổi nhằm giảm số lệnh mỗi vòng, thực hiện và đo lại ở x = 401; báo cáo N trước, N sau và phần trăm giảm. Nếu không giảm được, lập luận vì sao thân vòng đã tối thiểu.

Trả lời: Với biểu giá 60 bậc và x nằm bên trong bậc cuối, số lệnh dự kiến là N = 29 + 13 × 60 + 1 = 810 lệnh. Có thể tối ưu bằng cách lưu trực tiếp sức chứa của từng bậc vào mảng thay cho ngưỡng tích lũy, bỏ lệnh khởi tạo cận dưới và hai lệnh xử lý cận dưới trong mỗi vòng lặp. Với x = 401, số lệnh tính toán dự kiến giảm từ 108 xuống 95, tương đương khoảng 12,04%. Cần đối chiếu lại hai giá trị bằng Instruction Counter trên MARS.

Bước 4.2 – Giải mã lệnh của chương trình mình. Trong Text Segment của chương trình tiền điện, chọn ba lệnh: một lệnh nhánh trong vòng lặp, một lệnh nạp từ mảng và một lệnh so sánh. Với mỗi lệnh, ghi địa chỉ, mã hex, tách các trường và **tự đọc ra lệnh trước khi nhìn cột Basic**.

QUAN SÁT So khớp kết quả giải mã với cột Basic. Nếu khác, tìm trường nào bị đọc sai.

Bảng 16. Giải mã ba lệnh của chương trình tiền điện

Địa chỉ	Mã hex	Dạng lệnh	Các trường (tên = giá trị thập phân)	Lệnh đọc ra
0x00400064	0x11600001	I	opcode=4,rs=11,rt=0,immediate=1	beq $t3,$zero,tinh_tien
0x00400054	0x8e680000	I	opcode=35,rs=19,rt=8,immediate=0	lw $t0, 0($s3)
0x00400060	0x020a582a	R	opcode=0,rs=16,rt=10,rd=11,shamt=0, funct=42	slt $t3,$s0,$t2

Câu 4.3 (Áp dụng). Với lệnh nhánh đã chọn, tính (địa chỉ đích − (địa chỉ lệnh + 4)) / 4 và so với trường immediate trong mã hex. Với trường immediate 16 bit, một lệnh nhánh đến được các địa chỉ trong khoảng bao nhiêu byte so với lệnh kế tiếp?

Trả lời: Với lệnh beq tại địa chỉ 0x00400064 và nhãn đích tại 0x0040006C, immediate = [0x0040006C − (0x00400064 + 4)] / 4 = 1. Trường immediate 16 bit có dấu cho phép nhảy từ −32768 đến +32767 lệnh, tương đương −131072 đến +131068 byte tính từ PC + 4.

Bảng 17. Giả lệnh trong chương trình tiền điện

Giả lệnh trong mã nguồn	Lệnh cơ bản sinh ra	Số lệnh
la $a0,tb_nhap	lui, ori	2
la $s3,nguong	lui, ori	2
li $v0,4	addiu	1
li $s1,0	addiu	1
move $s0,$v0	addu	1

Câu 4.4 (Đánh giá). Chương trình của Sinh viên có bao nhiêu dòng lệnh nguồn và bao nhiêu lệnh cơ bản trong bộ nhớ? Nếu một giả lệnh nạp địa chỉ nằm trong thân vòng lặp thì nó ảnh hưởng thế nào đến N(x)? Nêu cách bố trí chương trình để tránh chi phí này.

Trả lời: Chương trình có 41 dòng lệnh nguồn trong phần .text và 47 lệnh cơ bản trong bộ nhớ. Sự khác biệt chủ yếu do 6 giả lệnh la được mở rộng thành 12 lệnh cơ bản. Nếu đặt giả lệnh nạp địa chỉ trong vòng lặp, số lệnh thực thi sẽ tăng theo số lần lặp. Để giảm chi phí, nên nạp địa chỉ mảng trước vòng lặp, lưu trong các thanh ghi con trỏ và chỉ cập nhật con trỏ khi duyệt.

TỔNG KẾT VÀ NỘP BÀI (20 phút)

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☐	File `Lab01_<MSSV>_dien.asm` Assemble không lỗi, có `.data`, `.text`, nhãn `main`, kết thúc bằng syscall 10 và chú thích vai trò thanh ghi.
☐	File `Lab01_<MSSV>_nhac.asm` Assemble không lỗi và in đúng khuôn các dòng yêu cầu.
☐	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp vào các khung nét đứt.
☐	Số liệu trong báo cáo khớp với chương trình nộp (đơn giá, tham số cá nhân và kết quả đo).
☐	Đặt tên file báo cáo: `Lab01_<MSSV>_BaoCao.docx`; nộp ba file theo hướng dẫn của giảng viên.

Tự chấm

Phần	Điểm tối đa	Tự chấm
TN 1	1,5	1,5
TN 2	4,0	4,0
TN 3	2,0	2,0
TN 4	2,0	2,0
Kết	0,5	0,5

Phản tư

Câu R1 (Đánh giá). Từ TN 1 và TN 4, Sinh viên rút ra điều gì về sự khác nhau giữa số dòng nguồn, số lệnh trong bộ nhớ và số lệnh thực thi? Nêu một tình huống cần quan tâm đến từng con số.

Trả lời: Số dòng nguồn là số lệnh người lập trình viết, số lệnh trong bộ nhớ là số lệnh cơ bản sau khi assembler chuyển đổi, còn số lệnh thực thi là số lệnh CPU thực sự chạy. Số dòng nguồn giúp đánh giá độ dài mã, số lệnh trong bộ nhớ giúp xác định dung lượng chương trình, còn số lệnh thực thi giúp đánh giá hiệu suất và thời gian chạy, đặc biệt với chương trình có vòng lặp.

Câu R2 (Phân tích). Từ TN 2, kiểu so sánh có dấu hay không dấu quyết định điều gì trong chương trình? Nêu một tình huống khác ngoài kiểm tra miền nhập mà chọn sai sẽ gây lỗi.

Trả lời: Kiểu so sánh có dấu hay không dấu quyết định cách CPU hiểu giá trị nhị phân. Trong TN2, dùng sltiu giúp kiểm tra được cả số âm và số lớn hơn 9999 chỉ bằng một phép so sánh. Nếu dùng slti, số -1 sẽ được chấp nhận vì -1 nhỏ hơn 10000, dẫn đến tính tiền sai. Một trường hợp khác là khi so sánh các địa chỉ bộ nhớ có bit cao nhất bằng 1, dùng so sánh có dấu có thể cho kết quả sai.

Câu R3 (Đánh giá). Nêu một lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời: Khi đo số lệnh thực thi ở TN4 với x = 120, em thấy Instruction Counter hiển thị 114 lệnh, khác với kết quả tính toán ban đầu. Qua kiểm tra, em nhận thấy có khả năng bộ đếm đã cộng dồn số lệnh từ các lần chạy trước. Vì vậy, cần Reset chương trình và Instruction Counter trước mỗi lần đo để kết quả chính xác. Qua đó, em rút ra bài học là phải kiểm tra trạng thái công cụ và đối chiếu kết quả thực tế với lý thuyết trước khi kết luận.

Tài liệu tham khảo

- Slide bài giảng Chương 2 – Kiến trúc bộ lệnh (Khoa Kỹ thuật Máy tính, UIT).
- MARS 4.5, Help ▸ Help: các mục Instructions, Pseudo-instructions, Syscalls.